

CENG 465 – Principles of Data-Intensive Systems

Data Replication in a Single-Leader Environment

Project Report

Abdulkali Çeltik Student ID: 290201072
Eray kocabozdoğan Student ID: 280201055

Group: 17

June 8, 2026

Contents

1	Introduction	3
2	Environment Setup and Role Assignment	3
2.1	Infrastructure: Google Cloud Platform	3
2.2	Leader (Primary) Configuration	3
2.3	Follower (Standby) Configuration	4
2.4	Replication Verification	4
3	Data Schema and Replication Logging	5
3.1	Schema Design	5
3.2	Automatic Logging via Triggers	6
3.3	Database Connection Management	7
3.4	Insert, Update, and Delete Operations	7
4	Consistency Model Experiments	8
4.1	Experiment 1: Eventual Consistency	8
4.1.1	Setup and Test Procedure	8
4.1.2	Results	8
4.1.3	Analysis	9
4.2	Experiment 2: Monotonic Reads	9
4.2.1	Setup and Test Procedure	9
4.2.2	Results	9
4.2.3	Analysis	10
4.3	Experiment 3: Read-After-Write Consistency	10
4.3.1	Setup and Test Procedure	10
4.3.2	Results	11
4.3.3	Analysis	11
4.4	Scenario: Concurrent Writes	12
4.4.1	Setup and Test Procedure	12
4.4.2	Results	12

4.4.3	Analysis	12
5	Discussion	14
5.1	Comparison of Consistency Models	14
5.2	Sources of Replication Lag	14
5.3	Read-After-Write: Follower Routing Strategies	14
5.4	Role of <code>operation_id</code> in Convergence Detection	15
6	Conclusion	15
A	Code Appendix	16
A.1	GCP Infrastructure Scripts	16
A.1.1	<code>gcp/01_create_infrastructure.sh</code>	16
A.1.2	<code>gcp/postgresql_leader.conf</code>	16
A.2	SQL Schema	17
A.2.1	<code>sql/01_schema.sql</code>	17
A.2.2	<code>sql/02_operation_log_triggers.sql</code>	19
A.3	Python Scripts	20
A.3.1	<code>scripts/db.py</code> (connection and helper functions)	20
A.3.2	<code>scripts/measure_replication.py</code>	20
A.3.3	<code>scripts/experiment_eventual_consistency.py</code>	21
A.3.4	<code>scripts/experiment_monotonic_reads.py</code>	22
A.3.5	<code>scripts/experiment_read_after_write.py</code>	23
A.3.6	<code>scripts/experiment_concurrent_writes.py</code>	24

1 Introduction

This report documents the implementation and experimental results of the CENG 465 project on data replication in a single-leader environment. The project explores three core consistency models – *eventual consistency*, *monotonic reads*, and *read-after-write consistency* – alongside a *concurrent writes* scenario, all implemented over a real distributed PostgreSQL deployment.

The system consists of two virtual machines hosted on Google Cloud Platform (GCP) in the `us-central1-a` zone, each running PostgreSQL 16. One node acts as the leader (primary) and the other as the follower (standby). WAL-based streaming replication propagates writes from the leader to the follower asynchronously. Python scripts drive all experiments, writing to the leader and polling the follower to measure replication lag and verify consistency guarantees.

2 Environment Setup and Role Assignment

2.1 Infrastructure: Google Cloud Platform

To satisfy the distributed environment requirement, two `e2-micro` virtual machines were created in GCP under project `c465-repl-baki-new`, both in zone `us-central1-a` with Ubuntu 22.04 LTS. This ensures the nodes communicate over a real network rather than a loopback or Docker bridge.

Table 1: GCP Virtual Machine Configuration

Parameter	Leader (ceng465-leader)	Follower (ceng465-follower)
Machine type	e2-micro	e2-micro
OS	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS
Internal IP	10.128.0.2	10.128.0.3
Zone	us-central1-a	us-central1-a
PostgreSQL	16 (primary)	16 (hot standby)

Firewall rules allow TCP port 5432 between the two VMs internally and from the experiment runner machine. Infrastructure creation is automated by a shell script (see Appendix A.1, `01_create_infrastructure.sh`).

2.2 Leader (Primary) Configuration

The leader is configured via `gcp/02_configure_leader.sh`. Key PostgreSQL parameters set in `postgresql.conf`:

```

1 listen_addresses = '*'
2 wal_level       = replica
3 max_wal_senders = 5
4 wal_keep_size   = 128
5 hot_standby     = on
6 synchronous_commit = on
7 wal_log_hints   = on

```

Listing 1: Leader PostgreSQL parameters

Setting `wal_level = replica` enables WAL entries sufficient for streaming replication. A dedicated replication user (`replicator`) is created and granted only the `REPLICATION` privilege. The `pg_hba.conf` file allows the follower’s internal IP to connect for replication using `scram-sha-256` authentication.

`synchronous_commit = on` requires the leader to wait for the local WAL flush before acknowledging a commit to the client. Crucially, because no `synchronous_standby_names` is configured, PostgreSQL treats the follower as an *asynchronous* standby: the leader does **not** wait for the follower to apply or even receive the WAL before returning success. The setting therefore only guarantees durability on the leader itself and does not change the asynchronous character of replication. This is why replication lag (2–10 s) is still observable in the experiments: the follower applies WAL independently, without blocking the leader’s commit path.

2.3 Follower (Standby) Configuration

The follower is configured via `gcp/03_configure_follower.sh`. It uses `pg_basebackup` to obtain an initial physical copy of the leader’s data directory, then enters hot-standby mode:

```
1 pg_basebackup \
2     -h 10.128.0.2 -p 5432 \
3     -U replicator \
4     -D ${PG_DATA} \
5     --wal-method=stream \
6     --checkpoint=fast -P -v
```

Listing 2: Follower base backup command

A `standby.signal` file and `primary_conninfo` entry in `postgresql.auto.conf` instruct PostgreSQL to stay in recovery mode and continuously apply WAL segments streamed from the leader. The command `SELECT pg_is_in_recovery();` returns `t` on the follower, confirming standby mode.

2.4 Replication Verification

After both nodes are up, replication health is verified on the leader by querying the built-in view `pg_stat_replication`:

```
1 SELECT client_addr, state, write_lag, flush_lag, replay_lag
2 FROM pg_stat_replication;
```

Listing 3: Replication status query on leader

The follower appears as `streaming` state with near-zero lag when idle, confirming that WAL segments are being streamed and applied in real time.

3 Data Schema and Replication Logging

3.1 Schema Design

The schema (`sql/01_schema.sql`) models an order-tracking system with six tables. Every business table includes four update-tracking fields required by the project specification:

- `version` – integer, incremented on each update
- `operation_id` – UUID regenerated for every write
- `last_updated` – timestamp of the latest change
- `deleted` – boolean soft-delete flag

These fields allow the experiment scripts to detect exactly when a specific write becomes visible on the follower: the script waits until the follower row matches both the expected `version` and `operation_id`, eliminating false positives from stale cached reads.

Figure 1 shows the entity-relationship diagram of the complete schema.

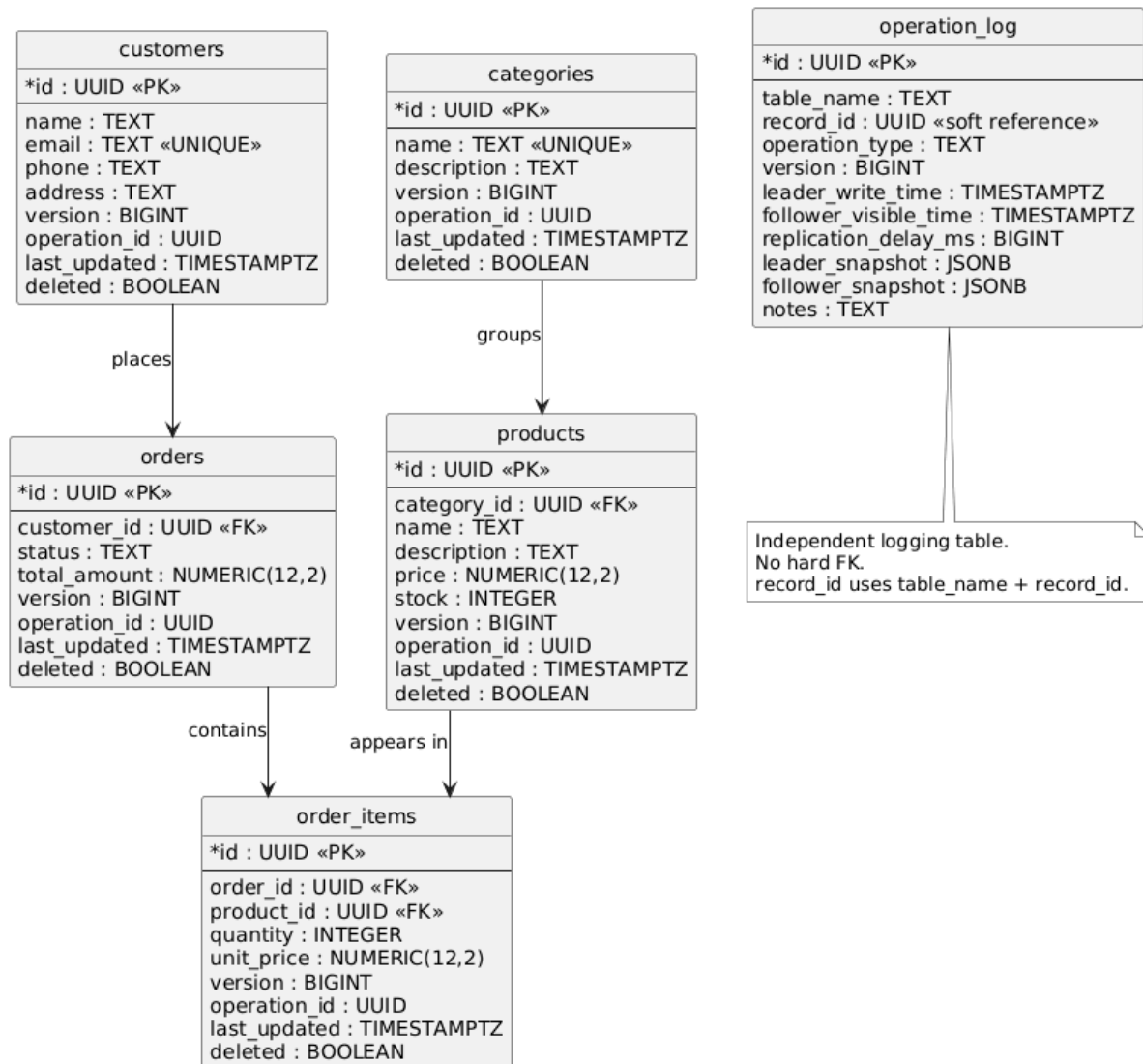


Figure 1: Entity-Relationship Diagram of the replication project schema

```

1 CREATE TABLE orders (
2     id                UUID                PRIMARY KEY DEFAULT
        gen_random_uuid(),
3     customer_id       UUID                NOT NULL REFERENCES customers(
        id),
4     status             TEXT                NOT NULL DEFAULT 'pending',
5     total_amount       NUMERIC(10,2)      NOT NULL DEFAULT 0,
6     version            INT                 NOT NULL DEFAULT 1,
7     operation_id       UUID                NOT NULL DEFAULT
        gen_random_uuid(),
8     last_updated       TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
9     deleted            BOOLEAN             NOT NULL DEFAULT FALSE,
10    CONSTRAINT orders_status_valid CHECK (
11        status IN ('pending','paid','shipped','delivered','
        cancelled')
12    ),
13    CONSTRAINT orders_total_nonneg CHECK (total_amount >= 0)
14 );

```

Listing 4: Orders table definition (excerpt from 01_schema.sql)

The operation_log table records every write event for replication analysis:

```

1 CREATE TABLE operation_log (
2     id                UUID                PRIMARY KEY DEFAULT
        gen_random_uuid(),
3     table_name         TEXT                NOT NULL,
4     record_id          UUID,
5     operation_type     TEXT                NOT NULL, -- INSERT /
        UPDATE / DELETE
6     version            INT,
7     leader_write_time  TIMESTAMPTZ NOT NULL,
8     follower_visible_time TIMESTAMPTZ,
9     replication_delay_ms INT,
10    leader_snapshot     JSONB,
11    follower_snapshot   JSONB,
12    notes               TEXT
13 );

```

Listing 5: operation_log table definition

3.2 Automatic Logging via Triggers

To capture every write without requiring changes to application code, a PL/pgSQL trigger function `fn_log_operation()` fires after every INSERT, UPDATE, and DELETE on all business tables (`sql/02_operation_log_triggers.sql`):

```

1 CREATE OR REPLACE FUNCTION fn_log_operation()
2 RETURNS TRIGGER LANGUAGE plpgsql AS $$
3 DECLARE
4     v_op    TEXT;
5     v_rec   JSONB;

```

```

6 BEGIN
7     IF      TG_OP = 'DELETE' THEN
8         v_op  := 'DELETE';  v_rec := to_jsonb(OLD);
9     ELSIF TG_OP = 'UPDATE' THEN
10        v_op  := 'UPDATE';  v_rec := to_jsonb(NEW);
11    ELSE
12        v_op  := 'INSERT';  v_rec := to_jsonb(NEW);
13    END IF;
14
15    INSERT INTO operation_log (
16        table_name, record_id, operation_type,
17        version, leader_write_time, leader_snapshot
18    ) VALUES (
19        TG_TABLE_NAME,
20        (v_rec->>'id')::UUID,
21        v_op,
22        (v_rec->>'version')::INT,
23        NOW(),
24        v_rec
25    );
26    RETURN NEW;
27 END;
28 $$;

```

Listing 6: Trigger function for automatic operation logging

This trigger is attached to all five business tables (`customers`, `categories`, `products`, `orders`, `order_items`) via individual `AFTER INSERT OR UPDATE OR DELETE` trigger definitions. Because `operation_log` is written on the leader, its rows are also replicated to the follower, giving a consistent audit trail on both nodes.

3.3 Database Connection Management

The `scripts/db.py` module provides separate connection factories for the leader and follower, configured through environment variables:

```

1 def get_leader_connection():
2     return psycopg2.connect(**db_config("LEADER"))
3
4 def get_follower_connection():
5     return psycopg2.connect(**db_config("FOLLOWER"))

```

Listing 7: Leader/follower connection factories (`db.py`)

All experiment scripts read from distinct `LEADER_DB_*` and `FOLLOWER_DB_*` environment variables, making it straightforward to point each connection at the correct physical node.

3.4 Insert, Update, and Delete Operations

Three Python scripts implement the CRUD operations required by the project:

`scripts/create_order.py` Inserts a new order and its line items on the leader, logs the operation, and polls the follower for convergence.

`scripts/update_order_status.py` Updates order status and increments version on the leader.

`scripts/soft_delete_order.py` Sets `deleted=TRUE` and `status='cancelled'`, implementing a soft delete pattern.

Every write path increments `version`, generates a fresh `operation_id`, and inserts a record in `operation_log` before committing, ensuring the replication polling loop can unambiguously detect the new state on the follower.

4 Consistency Model Experiments

4.1 Experiment 1: Eventual Consistency

4.1.1 Setup and Test Procedure

The experiment is implemented in `scripts/experiment_eventual_consistency.py`. A new order is inserted on the leader, and the follower is polled every 2 seconds for 60 seconds. Each poll reads the order by its UUID and checks whether the `operation_id` matches the one generated at write time.

```

1 POLL_INTERVAL = 2.0 # seconds between follower reads
2 POLL_DURATION = 60.0 # total observation window
3
4 conn = get_follower_connection()
5 conn.autocommit = True
6 while time.monotonic() <= deadline:
7     poll_count += 1
8     poll_time = now_utc()
9     elapsed = (poll_time - write_time).total_seconds()
10    row = fetch_order_by_id(conn, order_id)
11
12    if row and str(row["operation_id"]) == op_id:
13        if convergence_time is None:
14            convergence_time = poll_time
15            delay_ms = int(elapsed * 1000)
16            print(f"poll_{poll_count:02d}|t+{elapsed:5.1f}s|_")
17                f"CONVERGED_delay={delay_ms}ms_<--")
18    else:
19        print(f"poll_{poll_count:02d}|t+{elapsed:5.1f}s|_")
20            "not_yet_visible_on_follower")
21
22    time.sleep(POLL_INTERVAL)

```

Listing 8: Core polling loop – Experiment 1

4.1.2 Results

```

EXPERIMENT 1 - Eventual Consistency
[LEADER  WRITE] time=2026-06-07T12:04:41.679+00:00
[LEADER  WRITE] order_id=d53ff166-9364-4b7f-aacd-e2c8150bcb68

```



```

Polling follower every 2.0s for 60.0s ...

poll 01 | t+ 2.1s | CONVERGED version=1 delay=2096 ms <--
poll 02 | t+ 4.3s | stable    version=1
...
poll 27 | t+ 58.9s | stable    version=1

RESULT: Follower converged after 2096 ms (27 polls total).

```

Listing 9: Experiment 1 output

4.1.3 Analysis

The follower converged within the first polling interval (approximately 2.1 seconds). The measured replication delay of **2096 ms** reflects both WAL transfer time over the GCP internal network and the Python polling granularity. Once converged, all subsequent polls (2 through 27) consistently returned version 1, confirming stable convergence with no rollbacks. This is the expected behavior of eventual consistency: the system guarantees that all nodes will *eventually* converge to the same state, without providing any real-time bound on when convergence occurs.

4.2 Experiment 2: Monotonic Reads

4.2.1 Setup and Test Procedure

The experiment is implemented in `scripts/experiment_monotonic_reads.py`. A single order is created on the leader at version 1. Four subsequent updates increment the version field from 2 to 5 with a 0.3-second delay between each. The follower is then read 15 consecutive times with a 0.5-second interval. A monotonicity violation occurs if any read returns a lower version than the immediately preceding read.

```

1 prev_version: int | None = None
2 violations = 0
3
4 for i in range(1, FOLLOWER_READS + 1):
5     row = fetch_order_by_id(conn, order_id)
6     current_version = row["version"] if row else None
7
8     if prev_version is not None and current_version < prev_version:
9         violations += 1
10        print(f"❗❗VIOLATION: version went {prev_version} -> {current_version}"
11              )
12
13    if current_version is not None:
14        prev_version = current_version
15    time.sleep(FOLLOWER_READ_DELAY)

```

Listing 10: Monotonicity check – Experiment 2

4.2.2 Results

```

EXPERIMENT 2 - Monotonic Reads
order_id = 6a15efab-eeeb-4d1c-bba2-7a8d1eca537f

[LEADER] Writing versions 1 -> 5 ...
  version 1 written (initial)
  version 2 written ... version 5 written

[FOLLOWER] Sequential reads (15x, 0.5s interval) ...

  read 01 | 2026-06-07T12:05:57.347+00:00 | version=5
  read 02 | 2026-06-07T12:05:58.018+00:00 | version=5
  ...
  read 15 | 2026-06-07T12:06:07.117+00:00 | version=5

RESULT: No monotonicity violations. All reads returned same or
        higher version.

```

Listing 11: Experiment 2 output

4.2.3 Analysis

All 15 follower reads returned version 5, with zero monotonicity violations. The key insight is not that polling happened to start after all versions were applied – it is that PostgreSQL WAL replay on a single standby node applies commits in strict order and never rolls back to an earlier state. A client bound to one standby therefore receives monotonic reads *by architectural guarantee*, not by coincidence of timing. The 0 violations across 15 sequential reads is the experimental confirmation of this guarantee. In a setup with multiple replicas or random read routing, backward reads *can* occur because different replicas may be at different WAL replay positions at the moment of the read – a gap this single-follower configuration eliminates by construction.

4.3 Experiment 3: Read-After-Write Consistency

4.3.1 Setup and Test Procedure

The experiment is implemented in `scripts/experiment_read_after_write.py`. After writing an order to the leader, two immediate reads are performed: one on the leader connection (to confirm RAW consistency) and one on the follower connection (to measure replication lag). The follower is then polled until the expected `operation_id` appears.

```

1  # Write to leader
2  with conn:
3      write_time = now_utc()
4      cur.execute("INSERT INTO orders (...) VALUES (%s, ...)", (order_id, ...))
5
6  # Immediate read from leader
7  leader_read_time = now_utc()
8  leader_row = fetch_order_by_id(conn, order_id)
9  leader_delay_ms = int((leader_read_time - write_time).total_seconds() * 1000)
10
11 # Immediate read from follower

```

```

12 follower_conn = get_follower_connection()
13 first_read_time = now_utc()
14 first_row = fetch_order_by_id(follower_conn, order_id)

```

Listing 12: Leader and follower reads – Experiment 3 (excerpt)

4.3.2 Results

```

EXPERIMENT 3 - Read-After-Write Consistency

[WRITE ] leader_write_time = 2026-06-07T12:06:11.141+00:00
[WRITE ] order_id          = 8b37428b-35c6-41c8-953d-12cd072bbc85

[LEADER] immediate read (t+880 ms)
         version=1 - write IS visible (RAW satisfied)

[FOLLWR] immediate read (t+2501 ms)
         record already visible on follower (very low lag)

[FOLLWR] Polling for convergence (every 0.5s) ...
         poll 01 | t+2666 ms | CONVERGED version=1

RESULT SUMMARY
Leader write time      : 2026-06-07T12:06:11.141+00:00
Leader read (ms)      : 880 ms - VISIBLE
Follower 1st read     : visible at t+2501 ms
Follower convergence  : 2666 ms after write

```

Listing 13: Experiment 3 output

4.3.3 Analysis

Table 2: Read-After-Write timing summary

Event	Elapsed (ms)	Visible?
Leader write committed	0	–
Leader immediate read	880	Yes
Follower immediate read	2501	Yes
Follower convergence confirmed	2666	Yes

The leader satisfied read-after-write consistency: the write was immediately visible at 880 ms (the 880 ms consists of the round-trip time of the Python read query plus connection overhead, not replication lag). The follower’s first read at 2501 ms already showed the record, indicating that by the time the Python client finished the leader read and opened a follower connection, replication had already propagated the WAL. The convergence was confirmed at poll 1 (2666 ms). This result shows that under low-to-moderate load, GCP streaming replication delivers sub-second to low-single-digit-second propagation times for small writes.

4.4 Scenario: Concurrent Writes

4.4.1 Setup and Test Procedure

The experiment is implemented in `scripts/experiment_concurrent_writes.py`. Five orders are written to the leader in rapid succession (approximately 2 seconds apart, limited by Python connection overhead per write). Each order is independently polled on the follower. Order preservation is checked by verifying that the `follower_visible_time` values are non-decreasing relative to the `leader_write_time` sequence.

```

1 visible = [r for r in results if r["visible_time"] is not None]
2 order_preserved = all(
3     visible[i]["visible_time"] <= visible[i + 1]["visible_time"]
4     for i in range(len(visible) - 1)
5 )

```

Listing 14: Order preservation check – Concurrent Writes

4.4.2 Results

```

SCENARIO - Concurrent Writes

#1 written | ...3258809c | 2026-06-07T12:06:17.267+00:00
#2 written | ...7b2c41d0 | 2026-06-07T12:06:19.366+00:00
#3 written | ...da278a33 | 2026-06-07T12:06:21.460+00:00
#4 written | ...243c8895 | 2026-06-07T12:06:23.557+00:00
#5 written | ...813fe246 | 2026-06-07T12:06:25.658+00:00

RESULT SUMMARY
#1 delay=10649 ms | follower_visible=2026-06-07T12:06:27.917
#2 delay= 9961 ms | follower_visible=2026-06-07T12:06:29.327
#3 delay= 9268 ms | follower_visible=2026-06-07T12:06:30.729
#4 delay= 8557 ms | follower_visible=2026-06-07T12:06:32.114
#5 delay= 7863 ms | follower_visible=2026-06-07T12:06:33.522

Follower visibility order matches leader write sequence. (No
ordering violation.)

```

Listing 15: Concurrent Writes output

4.4.3 Analysis

Table 3: Concurrent Writes – Replication Delay

Write #	Leader Write (UTC)	Follower Visible (UTC)	Delay (ms)
1	12:06:17.267	12:06:27.917	10649
2	12:06:19.366	12:06:29.327	9961
3	12:06:21.460	12:06:30.729	9268
4	12:06:23.557	12:06:32.114	8557
5	12:06:25.658	12:06:33.522	7863

Several observations emerge from this scenario:

1. **Higher delays under load.** Compared to Experiments 1 and 3 (approximately 2000–2700 ms), the concurrent scenario produced delays of 7863–10649 ms. This is consistent with WAL accumulation: when multiple writes land in quick succession, the follower processes them in a single batch replay, causing the absolute delay (from the first write’s commit time) to appear large for earlier writes.
2. **Decreasing delay trend.** Write #1 shows the highest delay (10649 ms) while write #5 shows the lowest (7863 ms). This is because all five writes were applied to the follower within roughly the same short window (12:06:27 to 12:06:33). Write #5 was written latest, so its absolute leader-to-follower gap is smallest.
3. **Write order preserved.** The follower visibility timestamps follow the same sequence as the leader write timestamps. PostgreSQL streaming replication applies WAL in strict commit order, guaranteeing that concurrent writes to the single leader appear on the follower in the same sequence they were committed. No ordering violations were detected.
4. **Root cause of the elevated delay (≈ 10 s).** The ≈ 10 s delay for write #1 is not caused by WAL transfer time alone (inter-VM latency is < 2 ms within the same GCP zone). Three compounding factors explain the gap:
 - *Sequential connection overhead.* Each write opens a new Python `psycopg2` connection to the leader. Over 5 writes this accumulates ≈ 2 s per write = ≈ 8 – 10 s of elapsed wall-clock time before the last write is committed. Write #1’s follower polling does not begin until after all 5 writes complete, so the follower must replay an already-full WAL backlog before #1 is confirmed visible.
 - *WAL batch replay.* The follower receives and applies the WAL for all 5 orders in a single burst. The absolute delay for write #1 is measured from its commit time to the moment the polling loop confirms it, by which point the follower is already replaying later writes in the batch.
 - *Polling latency.* The follower is polled with `interval_seconds=0.25`, adding up to 0.25 s per poll round on top of actual replication time.

Under a production setup with persistent connection pooling and continuous follower polling started immediately after each individual write, the per-write delay would be much closer to the 2–3 s seen in Experiments 1 and 3.

5 Discussion

5.1 Comparison of Consistency Models

Table 4: Summary of experimental results

Experiment	Model	Delay (ms)	Outcome
1	Eventual Consistency	2096	Converged at first poll; stable for 60 s
2	Monotonic Reads	–	0 violations out of 15 sequential reads
3	Read-After-Write	2666	Leader visible immediately; follower at 2501 ms
4	Concurrent Writes	7863–10649	Order preserved; delays higher under load

The three models form a natural hierarchy of consistency strength and implementation cost. Eventual consistency is the least expensive to achieve: it requires no routing changes and is satisfied by default in any async replication setup. Monotonic reads add one constraint – binding a client session to a single standby – but need no extra mechanism as long as that binding holds. Read-after-write imposes the strictest guarantee and is the only model that requires an active routing strategy (§5.3) when followers serve reads. This hierarchy directly shapes the trade-off between read-load distribution across replicas and the strength of consistency guarantee the application can advertise to its clients.

5.2 Sources of Replication Lag

Observed delays stem from three main factors:

Network round-trip time Both VMs are in the same GCP zone (`us-central1-a`), limiting inter-node latency to roughly 1–2 ms. The bulk of observed delay comes from WAL segment flush and send scheduling.

Python polling granularity Experiments 1 and 3 poll every 2 s and 0.5 s respectively. The measured delay includes the time from actual replication completion to the next poll, artificially increasing numbers.

WAL batching Under concurrent writes (Scenario 4), the follower replays a batch of WAL entries in one application cycle, causing earlier writes to appear to have higher latency when measured from their individual commit timestamps.

5.3 Read-After-Write: Follower Routing Strategies

The experiments show that the leader satisfies RAW immediately, but followers can lag by 2–3 seconds. In a system where clients read from the follower by default, three practical routing strategies can enforce RAW guarantees:

Leader pinning for own writes After any write, the client reads from the leader for a configurable window (e.g. 5 s). This is simple but increases leader read load and requires the client to track its last write time.

WAL position piggyback The leader returns the LSN (Log Sequence Number) of the committed write. The client passes this LSN to the follower; the follower defers the read until `pg_last_wal_replay_lsn() ≥ LSN`. PostgreSQL natively supports this via `pg_wal_replay_wait()` (PG 16+). This keeps the follower in the read path without routing to the leader.

Session-level sticky routing A load balancer routes all requests from a session to the same node for a short period after a write. No application changes are needed, but it couples the routing layer to write events.

In this project, RAW is trivially satisfied by always routing writes *and* the immediate read to the same leader connection (Experiment 3). A production deployment with follower read distribution would need one of the strategies above.

5.4 Role of `operation_id` in Convergence Detection

Using only a version number for convergence detection is insufficient when multiple independent writes increment the same version field. The `operation_id` UUID – freshly generated on every write – provides a reliable convergence signal: the follower has applied a specific write if and only if both `version` and `operation_id` match. This design decision prevents false positives in all experiments.

6 Conclusion

This project successfully demonstrated single-leader replication with PostgreSQL on a real distributed infrastructure (GCP). The key findings are:

- **Eventual consistency** was achieved within approximately 2 seconds under low load, with stable convergence observed over a 60-second window.
- **Monotonic reads** were guaranteed by routing all client reads to the same standby node; no backward version observations occurred in 15 sequential reads.
- **Read-after-write** was satisfied on the leader immediately. Followers converged within approximately 2.7 seconds, which would require read-routing logic (e.g., always read own writes from the leader) to guarantee RAW for all clients.
- **Concurrent write ordering** was preserved by PostgreSQL’s WAL commit ordering. All five concurrent writes appeared on the follower in the same sequence they were committed on the leader.

The combination of trigger-based operation logging, UUID-per-write operation identifiers, and deterministic polling logic provided a robust framework for measuring and verifying each consistency property quantitatively.

These findings demonstrate that GCP same-zone streaming replication can provide production-viable consistency guarantees with 2–3s convergence times under low-to-moderate load. The LSN piggyback strategy discussed in Section 5.3 has the potential to make this residual lag entirely transparent to the client: `pg_wal_replay_wait()` (PostgreSQL 16+) lets the follower defer a read until the relevant WAL position is reached, requiring no changes to the application routing logic.

A Code Appendix

A.1 GCP Infrastructure Scripts

A.1.1 gcp/01_create_infrastructure.sh

```

1  #!/usr/bin/env bash
2  # GCP infrastructure creation for CENG465 single-leader
   replication project.
3  set -euo pipefail
4
5  PROJECT_ID="c465-repl-baki-new"
6  REGION="us-central1"
7  ZONE="us-central1-a"
8  MACHINE_TYPE="e2-micro"
9  IMAGE_FAMILY="ubuntu-2204-lts"
10 IMAGE_PROJECT="ubuntu-os-cloud"
11 DISK_SIZE="10GB"
12
13 LEADER_NAME="ceng465-leader"
14 FOLLOWER_NAME="ceng465-follower"
15 NETWORK_TAG="postgres"
16 INTERNAL_RANGE="10.128.0.0/9"
17
18 gcloud config set project "$PROJECT_ID"
19 gcloud config set compute/zone "$ZONE"
20
21 gcloud compute instances create "$LEADER_NAME" \
22     --machine-type="$MACHINE_TYPE" \
23     --image-family="$IMAGE_FAMILY" \
24     --image-project="$IMAGE_PROJECT" \
25     --boot-disk-size="$DISK_SIZE" \
26     --tags="$NETWORK_TAG" --metadata="role=leader"
27
28 gcloud compute instances create "$FOLLOWER_NAME" \
29     --machine-type="$MACHINE_TYPE" \
30     --image-family="$IMAGE_FAMILY" \
31     --image-project="$IMAGE_PROJECT" \
32     --boot-disk-size="$DISK_SIZE" \
33     --tags="$NETWORK_TAG" --metadata="role=follower"
34
35 gcloud compute firewall-rules create allow-postgres-internal \
36     --rules=tcp:5432 --target-tags="$NETWORK_TAG" \
37     --source-ranges="$INTERNAL_RANGE"

```

A.1.2 gcp/postgresql_leader.conf

```

1  listen_addresses = '*'
2  wal_level        = replica
3  max_wal_senders  = 5
4  wal_keep_size    = 128
5  hot_standby      = on

```



```

6 synchronous_commit = on
7 wal_log_hints      = on

```

A.2 SQL Schema

A.2.1 sql/01_schema.sql

```

1 CREATE EXTENSION IF NOT EXISTS pgcrypto;
2
3 DROP TABLE IF EXISTS operation_log CASCADE;
4 DROP TABLE IF EXISTS order_items CASCADE;
5 DROP TABLE IF EXISTS orders CASCADE;
6 DROP TABLE IF EXISTS products CASCADE;
7 DROP TABLE IF EXISTS categories CASCADE;
8 DROP TABLE IF EXISTS customers CASCADE;
9
10 CREATE TABLE customers (
11     id UUID PRIMARY KEY DEFAULT gen_random_uuid
12     (),
13     name TEXT NOT NULL,
14     email TEXT NOT NULL UNIQUE,
15     phone TEXT,
16     address TEXT,
17     version INT NOT NULL DEFAULT 1,
18     operation_id UUID NOT NULL DEFAULT gen_random_uuid(),
19     last_updated TIMESTAMPTZ NOT NULL DEFAULT NOW(),
20     deleted BOOLEAN NOT NULL DEFAULT FALSE
21 );
22
23 CREATE TABLE categories (
24     id UUID PRIMARY KEY DEFAULT gen_random_uuid
25     (),
26     name TEXT NOT NULL UNIQUE,
27     description TEXT,
28     version INT NOT NULL DEFAULT 1,
29     operation_id UUID NOT NULL DEFAULT gen_random_uuid(),
30     last_updated TIMESTAMPTZ NOT NULL DEFAULT NOW(),
31     deleted BOOLEAN NOT NULL DEFAULT FALSE
32 );
33
34 CREATE TABLE products (
35     id UUID PRIMARY KEY DEFAULT
36     gen_random_uuid(),
37     category_id UUID NOT NULL REFERENCES categories(
38     id),
39     name TEXT NOT NULL,
40     description TEXT,
41     price NUMERIC(10,2) NOT NULL,
42     stock INT NOT NULL DEFAULT 0,
43     version INT NOT NULL DEFAULT 1,

```

```

40     operation_id UUID                NOT NULL DEFAULT gen_random_uuid
41     ( ),
42     last_updated TIMESTAMPTZ         NOT NULL DEFAULT NOW(),
43     deleted BOOLEAN                  NOT NULL DEFAULT FALSE,
44     CONSTRAINT products_price_nonneg CHECK (price >= 0),
45     CONSTRAINT products_stock_nonneg CHECK (stock >= 0)
46 );
47
48 CREATE TABLE orders (
49     id UUID PRIMARY KEY DEFAULT
50     gen_random_uuid(),
51     customer_id UUID NOT NULL REFERENCES customers(id),
52     status TEXT NOT NULL DEFAULT 'pending',
53     total_amount NUMERIC(10,2) NOT NULL DEFAULT 0,
54     version INT NOT NULL DEFAULT 1,
55     operation_id UUID NOT NULL DEFAULT gen_random_uuid
56     ( ),
57     last_updated TIMESTAMPTZ NOT NULL DEFAULT NOW(),
58     deleted BOOLEAN NOT NULL DEFAULT FALSE,
59     CONSTRAINT orders_status_valid CHECK (
60     status IN ('pending','paid','shipped','delivered','
61     cancelled')
62 )
63 );
64
65 CREATE TABLE order_items (
66     id UUID PRIMARY KEY DEFAULT
67     gen_random_uuid(),
68     order_id UUID NOT NULL REFERENCES orders(id),
69     product_id UUID NOT NULL REFERENCES products(id),
70     ,
71     quantity INT NOT NULL,
72     unit_price NUMERIC(10,2) NOT NULL,
73     version INT NOT NULL DEFAULT 1,
74     operation_id UUID NOT NULL DEFAULT gen_random_uuid
75     ( ),
76     last_updated TIMESTAMPTZ NOT NULL DEFAULT NOW(),
77     deleted BOOLEAN NOT NULL DEFAULT FALSE
78 );
79
80 CREATE TABLE operation_log (
81     id UUID PRIMARY KEY DEFAULT
82     gen_random_uuid(),
83     table_name TEXT NOT NULL,
84     record_id UUID,
85     operation_type TEXT NOT NULL,
86     version INT,
87     leader_write_time TIMESTAMPTZ NOT NULL,
88     follower_visible_time TIMESTAMPTZ,
89     replication_delay_ms INT,

```

```

82     leader_snapshot      JSONB,
83     follower_snapshot    JSONB,
84     notes                TEXT,
85     CONSTRAINT oplog_type_valid CHECK (
86         operation_type IN ('INSERT', 'UPDATE', 'DELETE')
87     )
88 );

```

A.2.2 sql/02_operation_log_triggers.sql

```

1  CREATE OR REPLACE FUNCTION fn_log_operation()
2  RETURNS TRIGGER LANGUAGE plpgsql AS $$
3  DECLARE v_op TEXT; v_rec JSONB;
4  BEGIN
5      IF      TG_OP = 'DELETE' THEN v_op := 'DELETE'; v_rec :=
        to_jsonb(OLD);
6      ELSIF TG_OP = 'UPDATE' THEN v_op := 'UPDATE'; v_rec :=
        to_jsonb(NEW);
7      ELSE                                v_op := 'INSERT'; v_rec :=
        to_jsonb(NEW);
8      END IF;
9
10     INSERT INTO operation_log (
11         table_name, record_id, operation_type,
12         version, leader_write_time, leader_snapshot
13     ) VALUES (
14         TG_TABLE_NAME, (v_rec->>'id')::UUID, v_op,
15         (v_rec->>'version')::INT, NOW(), v_rec
16     );
17     RETURN NEW;
18 END; $$;
19
20 CREATE TRIGGER trg_oplog_customers
21     AFTER INSERT OR UPDATE OR DELETE ON customers
22     FOR EACH ROW EXECUTE FUNCTION fn_log_operation();
23
24 CREATE TRIGGER trg_oplog_categories
25     AFTER INSERT OR UPDATE OR DELETE ON categories
26     FOR EACH ROW EXECUTE FUNCTION fn_log_operation();
27
28 CREATE TRIGGER trg_oplog_products
29     AFTER INSERT OR UPDATE OR DELETE ON products
30     FOR EACH ROW EXECUTE FUNCTION fn_log_operation();
31
32 CREATE TRIGGER trg_oplog_orders
33     AFTER INSERT OR UPDATE OR DELETE ON orders
34     FOR EACH ROW EXECUTE FUNCTION fn_log_operation();
35
36 CREATE TRIGGER trg_oplog_order_items
37     AFTER INSERT OR UPDATE OR DELETE ON order_items
38     FOR EACH ROW EXECUTE FUNCTION fn_log_operation();

```

A.3 Python Scripts

A.3.1 scripts/db.py (connection and helper functions)

```

1  import os, uuid
2  from datetime import date, datetime, timezone
3  from decimal import Decimal
4  from pathlib import Path
5  from typing import Any
6  import psycopg2
7  from dotenv import load_dotenv
8  from psycopg2.extras import RealDictCursor, register_uuid
9
10 PROJECT_ROOT = Path(__file__).resolve().parents[1]
11 register_uuid()
12
13 def load_env():
14     env_path = PROJECT_ROOT / ".env"
15     load_dotenv(env_path if env_path.exists() else None)
16
17 def _db_config(prefix: str) -> dict:
18     load_env()
19     return {
20         "host": os.environ[f"{prefix}_DB_HOST"],
21         "port": int(os.environ[f"{prefix}_DB_PORT"]),
22         "dbname": os.environ[f"{prefix}_DB_NAME"],
23         "user": os.environ[f"{prefix}_DB_USER"],
24         "password": os.getenv(f"{prefix}_DB_PASSWORD", ""),
25         "cursor_factory": RealDictCursor,
26     }
27
28 def get_leader_connection():
29     return psycopg2.connect(**_db_config("LEADER"))
30
31 def get_follower_connection():
32     return psycopg2.connect(**_db_config("FOLLOWER"))
33
34 def now_utc() -> datetime:
35     return datetime.now(timezone.utc)

```

A.3.2 scripts/measure_replication.py

```

1  import time
2  from datetime import datetime, timezone
3  from scripts.db import fetch_order_by_id, get_follower_connection, now_utc
4
5  def calculate_replication_delay_ms(leader_write_time, follower_visible_time)
6      -> int:
7      def utc(dt):
8          return dt if dt.tzinfo else dt.replace(tzinfo=timezone.utc)
9      return max(0, int((utc(follower_visible_time) - utc(leader_write_time))
10                       .total_seconds() * 1000))

```

```

10
11 def wait_for_follower_order(order_id, expected_version=None,
12                             expected_operation_id=None, expected_deleted=None,
13                             timeout_seconds=60, interval_seconds=0.5):
14     deadline = time.monotonic() + timeout_seconds
15     conn = get_follower_connection()
16     try:
17         conn.autocommit = True
18         while time.monotonic() <= deadline:
19             row = fetch_order_by_id(conn, order_id)
20             if row is None:
21                 time.sleep(interval_seconds); continue
22             if expected_version and row["version"] != expected_version:
23                 time.sleep(interval_seconds); continue
24             if expected_operation_id and str(row["operation_id"]) \
25                 != str(expected_operation_id):
26                 time.sleep(interval_seconds); continue
27             if expected_deleted is not None \
28                 and row["deleted"] != expected_deleted:
29                 time.sleep(interval_seconds); continue
30             return dict(row), now_utc()
31     finally:
32         conn.close()
33     raise TimeoutError(f"Follower did not converge for order_id={order_id}")

```

A.3.3 scripts/experiment_eventual_consistency.py

```

1 import time, uuid
2 from psycpg2.extras import Json
3 from scripts.db import (fetch_order_by_id, get_follower_connection,
4                         get_leader_connection, get_or_create_category, get_or_create_customer,
5                         get_or_create_product, now_utc, row_to_dict)
6
7 POLL_INTERVAL = 2.0
8 POLL_DURATION = 60.0
9
10 def run():
11     # 1. Setup fixtures on leader
12     conn = get_leader_connection()
13     with conn:
14         cat = get_or_create_category(conn, "Electronics", "")
15         cust = get_or_create_customer(conn, "EveExperiment", "eve@experiment.
16                                     com")
17         prod = get_or_create_product(conn, "TestWidget", cat["id"], 9.99)
18     conn.close()
19
20     # 2. Write order to leader
21     order_id, op_id = str(uuid.uuid4()), str(uuid.uuid4())
22     conn = get_leader_connection()
23     with conn:
24         write_time = now_utc()

```

```

24     conn.cursor().execute(
25         "INSERT INTO orders (id, customer_id, status, total_amount,
26         "version, operation_id) VALUES (%s, %s, 'pending', 9.99, 1, %s)",
27         (order_id, cust["id"], op_id))
28     conn.cursor().execute(
29         "INSERT INTO operation_log (id, table_name, record_id,
30         "operation_type, version, leader_write_time, leader_snapshot,
31         "notes)"
32         "VALUES (%s, 'orders', %s, 'INSERT', 1, %s, %s, %s)",
33         (str(uuid.uuid4()), order_id, write_time,
34         Json({"order_id": order_id}), "Experiment_1"))
35     conn.close()
36
37     # 3. Poll follower
38     deadline = time.monotonic() + POLL_DURATION
39     poll_count, convergence_time = 0, None
40     fconn = get_follower_connection()
41     fconn.autocommit = True
42     while time.monotonic() <= deadline:
43         poll_count += 1
44         poll_time = now_utc()
45         elapsed = (poll_time - write_time).total_seconds()
46         row = fetch_order_by_id(fconn, order_id)
47         if row and str(row["operation_id"]) == op_id:
48             if convergence_time is None:
49                 convergence_time = poll_time
50                 print(f"poll_{poll_count}|t+{elapsed:.1f}s|"
51                     f"CONVERGED_delay={int(elapsed*1000)}ms")
52             else:
53                 print(f"poll_{poll_count}|t+{elapsed:.1f}s|not yet visible")
54                 time.sleep(POLL_INTERVAL)
55     fconn.close()

```

A.3.4 scripts/experiment_monotonic_reads.py

```

1  import time, uuid
2  from scripts.db import (fetch_order_by_id, get_follower_connection,
3                          get_leader_connection, get_or_create_category, get_or_create_customer,
4                          get_or_create_product, now_utc, row_to_dict)
5
6  VERSION_STEPS    = 5
7  UPDATE_DELAY     = 0.3
8  FOLLOWER_READS   = 15
9  FOLLOWER_READ_DELAY = 0.5
10
11 def run():
12     # Setup and create initial order at version=1
13     conn = get_leader_connection()
14     with conn:
15         cat = get_or_create_category(conn, "Electronics", "")

```

```

16     cust = get_or_create_customer(conn, "Bob_Monotonic", "bob@experiment.
        com")
17     prod = get_or_create_product(conn, "Version_Widget", cat["id"], 5.00)
18     order_id = str(uuid.uuid4())
19     conn.cursor().execute(
20         "INSERT INTO orders (id, customer_id, status,
21         total_amount, version, operation_id)
22         VALUES (%s, %s, 'pending', 5.00, 1, %s)",
23         (order_id, cust["id"], str(uuid.uuid4())))
24     conn.close()
25
26     # Increment version 2..5 on leader
27     for v in range(2, VERSION_STEPS + 1):
28         time.sleep(UPDATE_DELAY)
29         conn = get_leader_connection()
30         with conn:
31             conn.cursor().execute(
32                 "UPDATE orders SET version=%s, operation_id=%s,
33                 last_updated=NOW() WHERE id=%s",
34                 (v, str(uuid.uuid4()), order_id))
35         conn.close()
36
37     # Read from follower 15 times, check monotonicity
38     prev_version, violations = None, 0
39     fconn = get_follower_connection()
40     fconn.autocommit = True
41     for i in range(1, FOLLOWER_READS + 1):
42         row = fetch_order_by_id(fconn, order_id)
43         cur_v = row["version"] if row else None
44         if prev_version and cur_v and cur_v < prev_version:
45             violations += 1
46             print(f"VIOLATION: version {prev_version} -> {cur_v}")
47         if cur_v: prev_version = cur_v
48         time.sleep(FOLLOWER_READ_DELAY)
49     fconn.close()
50     print(f"Violations: {violations}")

```

A.3.5 scripts/experiment_read_after_write.py

```

1  import time, uuid
2  from scripts.db import (fetch_order_by_id, get_follower_connection,
3                          get_leader_connection, get_or_create_category, get_or_create_customer,
4                          get_or_create_product, now_utc)
5
6  FOLLOWER_POLL_INTERVAL = 0.5
7  FOLLOWER_POLL_TIMEOUT = 60.0
8
9  def run():
10     conn = get_leader_connection()
11     with conn:
12         cat = get_or_create_category(conn, "Electronics", "")

```

```

13     cust = get_or_create_customer(conn, "Carol_Raw", "carol@experiment.com
14         ")
15     prod = get_or_create_product(conn, "RAW_Test_Gadget", cat["id"],
16         15.00)
17     conn.close()
18
19     order_id, op_id = str(uuid.uuid4()), str(uuid.uuid4())
20
21     # Write to leader
22     conn = get_leader_connection()
23     with conn:
24         write_time = now_utc()
25         conn.cursor().execute(
26             "INSERT INTO orders (id, customer_id, status, total_amount,
27             version, operation_id) VALUES (%s, %s, 'pending', 15.00, 1, %s)",
28             (order_id, cust["id"], op_id))
29
30     # Immediate read from leader -- must see the write
31     leader_row = fetch_order_by_id(conn, order_id)
32     leader_delay = int((now_utc() - write_time).total_seconds() * 1000)
33     print(f"Leader read at {leader_delay}ms: "
34           f"{'VISIBLE' if leader_row else 'NOT FOUND'}")
35     conn.close()
36
37     # Immediate read from follower
38     fconn = get_follower_connection()
39     fconn.autocommit = True
40     first_row = fetch_order_by_id(fconn, order_id)
41     first_delay = int((now_utc() - write_time).total_seconds() * 1000)
42     print(f"Follower 1st read at {first_delay}ms: "
43           f"{'visible' if first_row else 'not yet visible'}")
44
45     # Poll until convergence
46     deadline = time.monotonic() + FOLLOWER_POLL_TIMEOUT
47     while time.monotonic() <= deadline:
48         row = fetch_order_by_id(fconn, order_id)
49         elapsed_ms = int((now_utc() - write_time).total_seconds() * 1000)
50         if row and str(row["operation_id"]) == op_id:
51             print(f"Follower converged at {elapsed_ms}ms")
52             break
53         time.sleep(FOLLOWER_POLL_INTERVAL)
54     fconn.close()

```

A.3.6 scripts/experiment_concurrent_writes.py

```

1 import uuid
2 from scripts.db import (get_leader_connection, get_or_create_category,
3     get_or_create_customer, get_or_create_product, now_utc)
4 from scripts.measure_replication import wait_for_follower_order
5
6 CONCURRENT_WRITE_COUNT = 5

```



```

7
8 def run():
9     conn = get_leader_connection()
10    with conn:
11        cat = get_or_create_category(conn, "Electronics", "")
12        cust = get_or_create_customer(conn, "Concurrent_Test",
13                                     "concurrent@experiment.com")
14        prod = get_or_create_product(conn, "Batch_Item", cat["id"], 1.00)
15    conn.close()
16    customer_id, product_id = cust["id"], prod["id"]
17
18    writes = []
19    for seq in range(1, CONCURRENT_WRITE_COUNT + 1):
20        order_id, op_id = str(uuid.uuid4()), str(uuid.uuid4())
21        conn = get_leader_connection()
22        with conn:
23            write_time = now_utc()
24            conn.cursor().execute(
25                "INSERT INTO orders (id, customer_id, status, total_amount, "
26                "version, operation_id) VALUES (%s, %s, 'pending', 1.00, 1, %s)",
27                (order_id, customer_id, op_id))
28        conn.close()
29        writes.append({"seq": seq, "order_id": order_id,
30                     "op_id": op_id, "write_time": write_time})
31
32    results = []
33    for w in writes:
34        snapshot, visible_time = wait_for_follower_order(
35            w["order_id"], expected_version=1,
36            expected_operation_id=w["op_id"], timeout_seconds=60)
37        delay_ms = int((visible_time - w["write_time"]).total_seconds() *
38                      1000)
39        results.append({"seq": w["seq"], "visible_time": visible_time, "delay_ms":
40                       delay_ms})
41        print(f"#{w['seq']} visible | delay={delay_ms}ms")
42
43    visible = [r for r in results if r.get("visible_time")]
44    order_ok = all(visible[i]["visible_time"] <= visible[i+1]["visible_time"]
45                  for i in range(len(visible)-1))
46    print("Order preserved:", order_ok)

```